

Documentation technique — Kasa

Ce document décrit l'installation, l'architecture, l'environnement de développement et les conventions du projet tels qu'ils existent aujourd'hui dans le dépôt.

1. Installation et lancement

```
yarn yarn start
```

yarn est le gestionnaire de paquets de ce projet (utilisé par le `Dockerfile`, seul lockfile versionné). N'utilisez pas `npm install/npm ci` : cela régénérerait un `package-lock.json` qui n'est plus suivi par git (ignoré volontairement, voir `.gitignore`) et pourrait diverger de `yarn.lock`.

Ouvrir <http://localhost:3000>, avec rechargement à chaud (Create React App) **et** recompilation automatique du Sass à chaque modification d'un fichier `.scss`.

Script	Commande	Rôle
<code>start</code>	<code>concurrently ... "npm:watch:css" "npm:start:react"</code>	Lance en parallèle le watcher Sass et le serveur de dev CRA
<code>start:react</code>	<code>react-scripts start</code>	Serveur de dev seul, sans watcher Sass
<code>watch:css</code>	<code>sass --watch main.scss:style.min.css</code>	Recompile le CSS à chaque sauvegarde d'un fichier <code>.scss</code>
<code>build</code>	<code>npm run build:css && react-scripts build</code>	Recompile le CSS puis build de production dans <code>build/</code> . Le <code>npm run</code> interne est volontaire (portabilité : <code>npm</code> est toujours présent avec Node, même sur une machine sans <code>yarn</code>) — lancer <code>yarn build</code> reste la commande recommandée (voir §1).
<code>build:css</code>	<code>sass src/styles/sass/main.scss src/styles/sass/style.min.css --style=compressed</code>	Compile une fois les sources Sass en CSS minifié

<code>test</code>	<code>react-scripts test</code>	Tests unitaires/intégration (Jest + React Testing Library), mode watch
<code>test:e2e</code>	<code>playwright test</code>	Tests end-to-end (Playwright), démarre automatiquement le serveur de dev
<code>lint</code>	<code>eslint src</code>	Analyse statique du code (config <code>react-app</code> déjà présente dans <code>package.json</code>)
<code>eject</code>	<code>react-scripts eject</code>	Sortie irréversible de CRA — aucune raison identifiée de l'utiliser

2. Structure des dossiers

```
public/ index.html # template HTML, charge Font Awesome via CDN logements.json # "back
kend" simulé – base de données des logements _redirects # règle SPA pour Netlify (voi
r docs/DEPLOYMENT.md §4) e2e/ navigation.spec.js # tests end-to-end Playwright src/ a
ssets/ fonts/, images/ # polices et images statiques components/ Card.jsx (+ .test.js
x) # carte logement (liste page d'accueil) Collapse.jsx (+ .test.jsx) # bloc dépliant
réutilisable, accessible au clavier Slideshow.jsx (+ .test.jsx) # carrousel photo d'
un logement layout/ Header.js, Footer.js pages/ Homepage.js (+ .test.js) About.js Err
or.js RentalDetails.js (+ .test.js) services/ rentals.js # accès centralisé aux donné
es (fetch + recherche par id) styles/ sass/ # sources Sass ET sortie compilée (style.
min.css) setupTests.js # config Jest/RTL (jest-dom, polyfills TextEncoder/TextDecoder)
App.js # définition des routes index.js # point d'entrée React Dockerfile, nginx.co
nf, .dockerignore # déploiement conteneurisé (build multi-stage + nginx) playwright.c
onfig.js
```

3. Technologies principales

- **React 19** (function components + hooks : `useState`, `useEffect`, `useParams`, `useNavigate`)
- **React Router v7** (`Routes/Route/Link`, `useParams`, `useNavigate`)
- **Sass**, compilé via le script `build:css` (voir §6)
- **Jest + React Testing Library** pour les tests unitaires/intégration (fournis par `react-scripts test`)
- **Playwright** pour les tests end-to-end
- **Font Awesome**, chargé via CDN dans `public/index.html` (pas une dépendance npm)
- **Docker + nginx** pour le déploiement (voir §7)

4. Architecture de l'application

SPA classique CRA : `index.js` monte `<App />` dans `<BrowserRouter>`.

`App.js` définit 4 routes :

- `/` → `Homepage`
- `/a-propos` → `About`
- `/location/:id` → `RentalDetails`
- `/*` → `Error` (catch-all, y compris ID de logement inconnu)

Chaque page compose ses propres `Header/Footer`. La navigation interne utilise `<Link>` (pas de rechargement de page). L'accès aux données passe désormais par une couche unique (`src/services/rentals.js`), consommée par `Homepage`, `RentalDetails` et `Slideshow` — plus de `fetch` dupliqué. Aucune gestion d'état globale (pas de Redux/Context) : chaque page garde son propre `useState` local.

5. Fonctionnement du "faux backend" JSON

- Fichier : `public/logements.json`, servi statiquement (CRA en dev, nginx en prod) à l'URL relative `/logements.json`.
- Structure de chaque entrée : `id`, `title`, `cover`, `pictures[]`, `description`, `host: {name, picture}`, `rating` (string "0"- "5"), `location`, `equipments[]`, `tags[]`.
- `src/services/rentals.js` expose :
- `getRentals()` : fetch le fichier, lève une erreur explicite si la réponse n'est pas `ok`.
- `findRentalById(rentals, id)` : filtrage côté client par `Array.find`.
- Les composants consommateurs stockent le résultat en `useState`, gèrent un état d'erreur affiché à l'utilisateur (plus seulement `console.error`), et pour `RentalDetails`, redirigent vers `/error` si l'ID ne correspond à aucune entrée.

6. Pipeline Sass

Les partiels sources vivent dans `src/styles/sass/` et référencent les assets avec des chemins relatifs à **deux niveaux sous `src/`** (ex. `url(../../assets/images/...)`). Les scripts `build:css/watch:css` compilent donc `main.scss` directement vers `src/styles/sass/style.min.css` (et non vers `src/styles/style.min.css`), pour que ces chemins résolvent correctement. `index.js` importe ce fichier compilé.

`yarn start` lance automatiquement le watcher (`sass --watch`, via `concurrently`) en plus du serveur CRA : toute modification d'un `.scss` recompile `style.min.css` immédiatement, sans commande manuelle. Le build de production (`yarn build`) recompile aussi le CSS avant de builder, pour ne jamais partir d'une version obsolète.

7. Déploiement (Docker)

Le `Dockerfile` est un build multi-stage :

1. **Stage `build`** (Node 22) : installe les dépendances (`yarn install --immutable`) puis exécute `yarn build`.
2. **Stage `runtime`** (nginx alpine) : copie `build/` dans `/usr/share/nginx/html` et utilise `nginx.conf` (fallback SPA via `try_files ... /index.html`).

```
docker build -t kasa . docker run -p 8080:80 kasa
```

8. Conventions importantes

- Composants réutilisables en `.jsx` (`Card.jsx`, `Collapse.jsx`, `Slideshow.jsx`), pages/layout en `.js`.
- Nommage PascalCase pour les composants.
- Navigation interne : toujours `<Link>` de react-router, jamais `<a href>` (sauf liens externes).
- Accessibilité : `aria-label`, `tabIndex="0"`, `sr-only`, `aria-hidden`, gestion clavier (`onKeyDown` sur les éléments interactifs comme le chevron de `Collapse`) — convention forte à respecter dans tout nouveau composant.
- Accès aux données : toujours passer par `src/services/rentals.js`, ne pas réintroduire de `fetch('/logements.json')` direct dans un composant.
- Tests colocalisés : `Composant.test.jsx`/`Page.test.js` à côté du fichier testé (détecté automatiquement par `react-scripts test`) ; tests e2e dans `e2e/`.

9. Développer, tester et maintenir

- **Développer** : `yarn start`, éditer dans `src/`. Pour les styles, éditer les `.scss` puis `yarn build:css`.
- **Tester** : voir [docs/TESTING_STRATEGY.md](#) pour le détail (outils, priorités, organisation, commandes).
- **Maintenir les données de démo** : éditer `public/logements.json` en respectant le schéma existant (§5).
- **Lint** : `yarn lint` avant de commit.

10. Points connus, non résolus (hors périmètre des correctifs appliqués)

- Les sources Sass utilisent encore `@import`, marqué déprécié par Dart Sass (migration vers `@use/@forward` non faite) — le watcher affiche ces avertissements à chaque compilation sans bloquer.
- `Header.js`, `Footer.js` et `About.js` n'ont pas encore de tests dédiés.
- Le dépôt GitHub a été renommé (`kasa` → `Kasa-React`) ; l'URL du remote local devrait être mise à jour (`git remote set-url origin ...`).